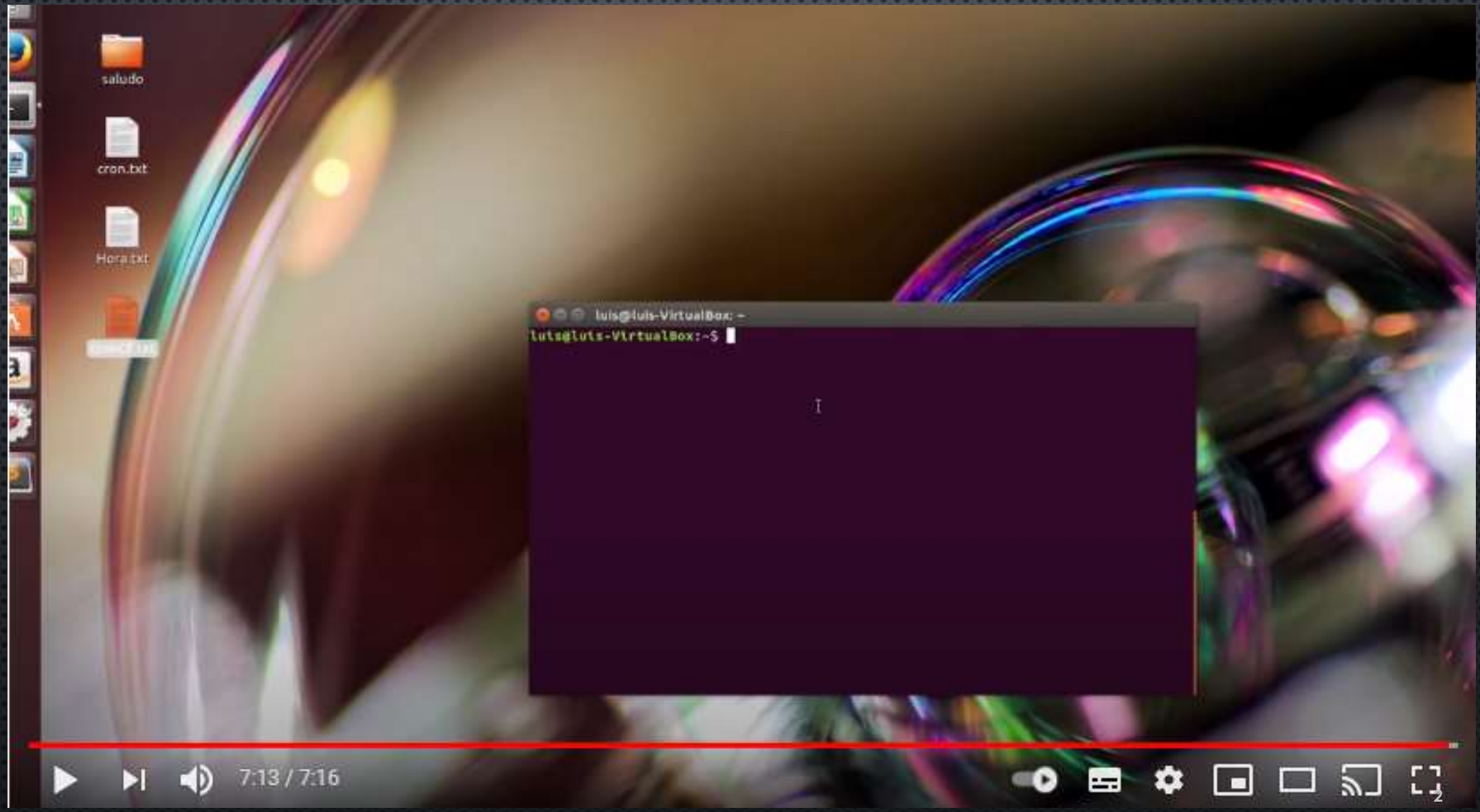
CRON & CRON TAB

HAZEM ÁLVAREZ RODRÍGUEZ



https://www.youtube.com/watch?v=lvW6Fw3Ghy8



CRON

Nombre del *programa que permite a usuarios* Linux/Unix **ejecutar automáticamente comandos o scripts** (grupos de comandos) a una hora o fecha específica.

Es usado normalmente para comandos de tareas (*jobs*) administrativas, como respaldos, pero puede ser usado para ejecutar varias tareas.

Como se define en las páginas del manual de cron (#> man cron) es un demonio que ejecuta programas agendados. En función de la distribución, se inicia utilizando las carpetas ***/etc/rc.d/*** o ***/etc/init.d*** y ***cada minuto comprueba los ficheros /etc/crontab o /var/spool/cron*** es busca de posibles ejecuciones (ahora empieza todo a cuadrar).



En prácticamente todas las distribuciones de Linux se usa la versión Vixie Cron, por la persona que la desarrolló, que es Paul Vixie, uno de los grandes gurús de Unix, también creador, entre otros sistemas, de BIND que es uno de los servidores DNS más populares del mundo.

Recuperado de https://migueltuyare.ar/phocadownload/manuales/ManualBasicodeCron.pdf

INICIAR CRON

Cron es un demonio (servicio), lo que significa que solo requiere ser iniciado una vez, generalmente con el mismo arranque del sistema.

El servicio de cron se llama crond. En la mayoría de las distribuciones el servicio se instala automáticamente y queda iniciado desde el arranque del sistema, se puede comprobar de varias maneras:



De acuerdo a la distro se puede emplear:

#> /ETC/RC.D/INIT.D/CROND STATUS

#> /ETC/INIT.D/CROND STATUS

En caso de tener Service instalado

#> SERVICE CROND STATUS

O por comando ps

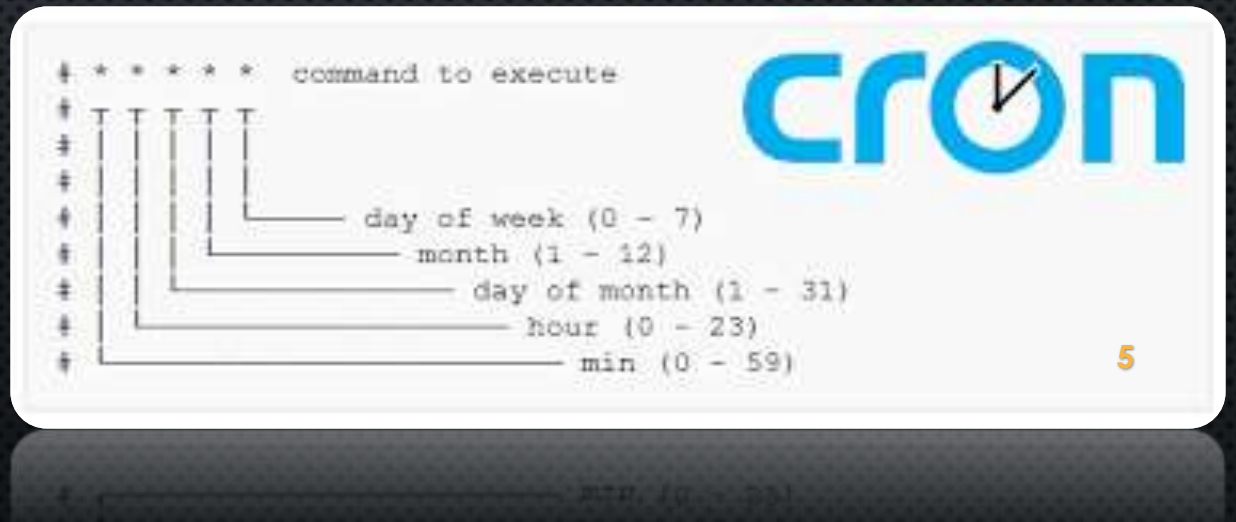
PS -EF | GREP CROND

CRON TAB

Un archivo de texto. Por simple que parezca la definición es así. Aunque es verdad que se trata de un archivo con contenido especial.

Posee una lista con todos los scripts a ejecutar. Generalmente cada usuario del sistema posee su propio fichero Crontab. Se considera que el ubicado en la carpeta ***etc/*** pertenece al usuario *root*.

Para generar el archivo propio, cada usuario deberá hacer uso del comando crontab (sí, es el mismo nombre).



OPCIONES CRONTAB



- ✓ Editar el archivo Crontab , ***$ crontab -e***
- ✓ Listar todas las tareas existentes en el crontab del usuario, ***$ crontab –l***
- ✓ Borrar el crontab que está configurado, ***$ crontab -d***

AGREGAR TAREAS A CRONTAB

1. Cree un script Shell bash sencillo
2. Guarde con extensión .sh y proporcione permisos de ejecución
3. Agregue la ruta del script al crontab, indicando los días, horas, minutos, mes, etc.

Ejecutar todos los días a las 7 de la tarde →*00 19 * * * USUARIO/UBICACION/DEL/SCRIPT/CONSULTA.SH*

Ejecutar el script todos los 4 de febrero a las 7 de la tarde → 00 19 4 2 * usuario /ubicacion/del/script/consulta.sh

SHELL & SCRIPT

¿QUÉ ES UN SHELL?

- Es un programa que tiene como función la de ***proporcionar la interfaz entre el usuario y el sistema Unix -y el núcleo*** *(kernel)* del sistema GNU/Linux. O también considerado *como* ***intérprete de comandos***.
- El shell es lo que los usuarios ven del sistema y está escrito de la misma forma que un proceso (programa) de usuario, **no está integrado en el kernel**, **sino que se ejecuta como un programa más del usuario**.
- Permite a los usuarios interactuar con el kernel por interpretación de comandos. Los comandos son los que usuario introduce en la línea de comandos o en los ficheros de tipo ***Shell script***.



CARACTERÍSTICAS

Los intérpretes de comandos disponibles en los entornos Unix tienen en común las siguientes funcionalidades:

- Proponen un juego de caracteres especiales que permiten desencadenar acciones concretas.
- Tienen comandos internos y palabras clave mediante algunos de los cuales se puede programar.
- Utilizan archivos de inicialización que permiten a un usuario parametrizar su entorno de trabajo.

Cada shell **propone sus propios caracteres especiales**, **comandos internos**, **palabras clave** y **archivos de configuración**. Afortunadamente, los intérpretes de comandos actualmente más utilizados derivan todos del *shell Bourne* y tienen, por consiguiente, un cierto número de funcionalidades en común.



Recuperado de https://www.ediciones-eni.com/open/mediabook.aspx?idR=6ff1f02359931c4aa334bc676125e3ca

TIPOS DE SHELL

- ***Bourne Shell (sh)***. Escrito en los años 70 por ***Steve Bourne*** en los laboratorios AT&T. Además de ejecutar comandos, dispone de funcionalidades de programación.
- ***Bill Joy*** inventa el ***C-Shell (csh)***, incompatible con Bourne, pero ofrece funcionalidades suplementarias como el histórico de comandos, el control de tareas y la posibilidad de crear alias de comandos. Estos tres aspectos se retomarán más tarde en el ***Korn Shell***.
- ***C-shell*** es minimante usado en el mundo Unix.
- En 1983, **David Korn** retoma el *Bourne Shell* y lo enriquece. Este nuevo intérprete tomará el nombre de ***Korn Shell (ksh) y será*** cada vez más usado hasta el punto de convertirse en un estándar. El ***ksh 88*** (versión de 1988) y ***Bourne Again Shell es el más*** utilizado y utlizado para la estandarización del shell (IEEE Posix 1003.2).
- En 1993, una nueva versión de ***Korn Shell*** ve la luz. Presenta una retrocompatibilidad con ksh 88, con algunas excepciones. ksh 93 (/usr/bin/ksh93) está disponible de forma estándar en algunas versiones UNIX recientes: Solaris 11, AIX 6 y 7.
- La Free Software Foundation propone el ***Bourne Again Shell (bash***). Sigue la normativa POSIX a la que ha añadido algunas extensiones.



Recuperado de https://www.ediciones-eni.com/open/mediabook.aspx?idR=6ff1f02359931c4aa334bc676125e3ca

CONTINUACIÓN…

Unix tiene 2 categorías principales de shells.

Shell tipo Bourne o Bourne shell se clasifica además como:

- ✓ Bourne shell (sh)
- ✓ Korn shell (ksh)
- ✓ POSIX shell (sh)
- ✓ Bourne Again shell (bash)

C shell también se clasifica además como:

- C shell (csh)
- TENEX (TOPS) C shell (tcsh)





Recuperado de https://www.hostinger.mx/tutoriales/bash-script-linux

SHELL SCRIPT

En la automatización de tareas de administración suelen utilizarse comandos agrupados en **shell scripts** (*también llamados* ***guiones de comandos***), **lenguajes interpretados por el shell** (*intérprete de comandos*) **del sistema**, y nos **permiten unir los comandos del sistema con estructuras de control de flujo** y disponer de un entorno de prototipo rápido de herramientas para la automatización de tareas.

La automatización de tareas pueden no ser adecuadas a actividades que requieran ejecutar pasos especialmente definidos, por lo que no están pensadas hacia las actividades o pasos a ejecutar o bien no ser eficaz.

SCRIPTS SHELL BASH

HTTPS://LIKEGEEKS.COM/ES/SCRIPT-DE-BASH-TUTORIAL/

hazem@haz: ~

```
 GNU nano 2.9.3                                  uno.sh

#! /bin/bash

echo "Este es un comentario"

pwd

whoami
```

```
hazem@haz:~$ nano uno.sh
hazem@haz:~$ ./uno.sh
Este es un comentario
/home/hazem
hazem
hazem@haz:~$
```

REFERENCIAS Y FUENTES DE INFORMACIÓN

1. Gonzalez S. Manual BÁsico de Cron, El demonio para backups,GNU Free,©2005-2008 recuperado de https://migueltuyare.ar/phocadownload/manuales/ManualBasicodeCron.pdf
2. Programación shell en Unix/Linux - sh, ksh, bash [2ª edición] en https://www.ediciones-eni.com/open/mediabook.aspx?idR=e9e746f52f418659a2e40ff19a3c5683
3. Curso de introducción a GNU/Linux – Crontab en https://www.youtube.com/watch?v=lvW6Fw3Ghy8
4. Comando date, sus usos y respaldo de archivos en https://www.linuxtotal.com.mx/index.php?cont=info_admon_007
5. Programa de archivo tar: creación de backups y archivado de ficheros en https://www.ionos.mx/digitalguide/servidores/herramientas/programa-de-archivo-tar-como-crear-backups-en-linux/
6. Como hacer un shell script en Linux en https://www.profesionalreview.com/2017/03/12/shell-script-linux/
7. CÓmo usar el comando Tar en Linux en https://www.hostinger.es/tutoriales/como-usar-comando-tar-linux

GRACIAS!!!